Performance Comparison of an ABI-Aware Approach to Access Structures in Java

Linus Andera
Faculty Computer Science
Hochschule Coburg
Coburg, Germany
linus.andera@stud.hs-coburg.de

***Abstract*—Efficient access to native structures is essential for performance-critical Java applications, such as real-time rendering. Existing libraries - such as LWJGL, the Foreign Function & Memory API (FFMA) and JNA - facilitate this but do not provide support for Application Binary Interfaces (ABI) like OpenGL's Standard Uniform Block Layout (std140). That is why this paper introduces an ABI-aware approach named LUtils to access native structures in Java. A microbenchmark study using JMH was conducted to compare LUtils with LWJGL, FFMA and JNA in terms of structure creation, memory allocation and write/read operations. The evaluation reveals that LUtils – in its current form – should not be used for performance-critical applications. Instead, the libraries LWJGL or FFMA should be preferred.**

***Index Terms*—java, native access, structure, jmh, performance, lwjgl, jna, ffma**

I. Introduction

The high-level programming language Java does not support simple aggregate types such as C-style structures (struct) or unions [1]. Consequently, a variety of libraries have been developed to bridge this gap and enable interaction with native memory layouts [2]. This study introduces a novel approach to dynamically create aggregate types from special Java classes at runtime, while providing support for different Application Binary Interfaces (ABI). Especially in performance critical applications, like real-time rendering, different ABIs are required to transfer structures to graphic processors using Vulkan[1], OpenGL[2] or OpenCL[3] . These ABIs are not supported by current Java native interop libraries. LUtils includes multiple ABIs and provides the developer with the ability to implement custom ABIs. However, it has not seen much usage in real-world applications and is missing comparisons to already existing libraries in terms of performance and memory usage. Therefore, this paper aims to compare the performance of LUtils against existing solutions – namely Lightweight Java Game Library (LWJGL), Foreign Functions and Memory API (FFMA) and Java Native Access (JNA) – in terms of structure creation and write/read operations. Thus, answering whether LUtils can be used in performance-critical applications or if other libraries should be preferred.

[1]https://docs.vulkan.org/spec/latest/chapters/interfaces.html#interfaces-resources-layout

[2]https://registry.khronos.org/OpenGL/specs/gl/glspec46.core.withchanges.pdf

[3]https://ieeexplore.ieee.org/abstract/document/7478342/

There have not been many studies analysing performance of Java native interop libraries with respect to structures. However, one remotely related approach [3] aims to improve memory performance of embedded Java applications. It discusses dynamically changing the memory layout of arrays on embedded systems to improve performance. Other research [4]–[6] discusses common practices and security related issues of the Java Native Interface (JNI). Another study [7] evaluates the performance of the Java Vector API in vector embedding operations, comparing it to pure Java solutions and C++ implementations called from Java.

Another related study [8] discusses and compares tools that asses ABI compatibility. These tools predict ABI incompatibility and detect potential bugs arising from ABI mismatches. Furthermore, the work highlights reasons why ABI compatibility can be problematic and how an understanding of ABIs might be required to solve ABI related problems.

The work most closely related to this paper is the bachelor's thesis by Niklas Seppälä [9],which investigates whether Java native interop can improve performance compared to purely Java-based implementations. That study focuses on JNI, JNA and FFMA, concluding that utilizing Java native interop is most suitable for larger tasks, which minimize communication between Java and native code. This represents the key difference to the approach presented by the present paper, which aims to analyse the impact of creating and accessing structures in Java code without passing the structures to native code.

The remainder of this article is organised as follows: The next section provides a concise introduction to LUtils, followed by a description of the experimental methodology. Subsequently, the results of the experiments are presented and discussed. Finally, the paper concludes with a summary of the findings.

II. LUtils

LUtils provides the class `Structure` from which every other structure must extend. Furthermore, LUtils contains many already implemented structures. This includes wrapper classes for primitive types, such as the class `BBInt1`, which represents a 32-bit integer. Additionally, 2- to 4-component vector types are provided (`BBFloat2`, `BBFloat3`, `BBFloat4`) as well as different array types. These include the class `StructureArray` representing an

array of any structure as well as special classes for arrays of primitive types as well as utf-8 and utf-16 strings.

Developers can use these types to implement C-style structures (`struct`) by extending the `ComplexStructure` class. For example, a structure containing a 32-bit integer, an 8-bit integer and a 64-bit integer would be implemented as shown in Listing 1.

Listing 1. Example structure implemented in LUtils

```
public class SmallTestStruct2 extends
    ComplexStructure {
  public final @StructValue(0) BBInt1 aInt =
      BBInt1.newUnallocated();
  public final @StructValue(1) BBByte1 aByte =
       BBByte1.newUnallocated();
  public final @StructValue(2) BBLong1 aLong =
       BBLong1.newUnallocated();

  public SmallTestStruct2() { super(false); }
}
```

LUtils uses Java reflection to retrieve the `StructureLayoutSettings` annotation from the class extending `ComplexStructure`. This annotation defines the ABI to use for the structure. However, if the annotation is not present Microsoft's x64 ABI convention is used by default. After the ABI has been selected all fields of the structure are listed using reflection and the memory layout is calculated. Subsequently, native memory can be allocated by calling `allocate()`, which internally uses `ByteBuffer.allocateDirect()`.

III. Methodology

Three experiments are performed to compare LUtils to LWJGL, FFMA and JNA. The experiments measure structure creation and write/read performance for structures of different sizes and complexities. The Java Microbenchmark Harness (JMH) is used to measure execution time and Java-Heap allocation rate across multiple benchmarks. More specifically, each experiment features two benchmarks, which are executed separately for each library. To ensure reproducibility the code and results of each experiment are available on GitHub[4].

The first benchmark of each experiment aims to compare the native memory allocation and initialization of classes required for the structures. No elements of the structures are set to any value or read from. The benchmark is intentionally minimalistic to isolate overhead associated with memory allocation and object creation.

The goal of the second benchmark is to compare the performance of writing to and reading from structures. At least one structure is allocated, and all fields are set to random values. The random values - generated using `java.lang.Random` with a fixed seed - are the same for each of the four libraries. Afterwards all fields are read and compared with the original input. The comparison code is the same for all four libraries, thus not skewing the benchmarks. Additionally, a reference benchmark is run which allocates the same structures but without performing any read or write operations, thereby isolating the overhead introduced by memory access. This reference benchmark is displayed in the resulting plots.

All benchmarks are executed on the same hardware, which is a Dell Latitude 5300 laptop with an Intel Core i5 vPro CPU with 16 Gigabyte of DDR4 RAM and no dedicated GPU. The laptop is running the operating system Pop!_OS 24.04 LTS. Additionally, Intel SpeedStep, Intel Turbo Boost, hyperthreading and address space layout randomisation have been disabled to avoid performance fluctuations caused by these features [13]. The available memory has been fixed to 8GB using Java's `Xmx` flag.

Furthermore, common benchmark pitfalls [10], [11] have been avoided by using the respective JMH features:

Dead code elimination:
: To avoid dead code elimination all unused variables are consumed using a `BlackHole` instance.

Constant folding:
: To avoid constant folding all constants are stored in non-final class variables.

Wrong stable state:
: To achieve a stable state 50 warmup iterations are included in every benchmark

Each iteration executes the benchmark code repeatedly for one second and every benchmark consists of 50 warmup and 400 measurement iterations, which provide at least 4,000 invocations of benchmark code. Additionally, the measurements are run three times (3 forks) on fully clear instantiations of new JVMs. These show that the medians vary less than 5% across all three forks. Except in benchmark 2 of experiment 3, where FFMA exhibits a median variation of 8.29%.

A. Experiment 1

The first experiment evaluates small structures using the four different java native access libraries: LUtils, FFMA, JNA and LWJGL. The structures all have a sizes between 8 bytes and 48 bytes. Additionally, they contain up to three different element types. Even though each element can be another structure containing more element types, the size constraint retains a low amount of different element types. Notably, the structure with the greatest complexity contains five different element types.

The first benchmark of this experiment allocates ten different structures without reading or writing to them. This provides variation in memory layouts, which helps to capture a broader range of real-world usage scenarios. Furthermore, it reduces the risk of bias caused by library specific optimisations or inefficiencies. The second benchmark allocates two different structures to write to and read from. Only two structures are used, because each structure provides multiple read and write operations.

B. Experiment 2

The second experiment evaluates medium structures across the four libraries. The structures have sizes between 112 bytes

[4]https://github.com/lni-dev/thesis-lutils

and 424 bytes containing between five and ten different element types. Once again, each element can be another structure which may contain more element types. Thus, this experiment aims compare structure creation, writing and reading with more complex structures.

The first benchmark of this experiment allocates five different structures without reading or writing to them. Due to the much higher complexity of the structures, this provides variation in memory layout and reduces the risk of bias caused by library specific optimizations or inefficiencies. In contrast, the second benchmark allocates only a single structure to write to and read from. A single structure provides enough variation, because of the high complexity of the structure itself and the many read and write operations required to write to every field of the structure.

C. Experiment 3

The third experiment evaluates large structures across all four libraries. The sizes of these structures range from 496,000 bytes to 1,184,000 bytes. The structures mostly contain arrays of primitive types or arrays of other structures. Thus, this experiment's goal is to compare structure creation, writing and reading with very large and complex structures with a lot of arrays.

Once again, the first benchmark allocates five different structures without reading or writing to them. Five structures are used to provide variation in memory layout and reduce the risk of bias caused by library specific optimizations or inefficiencies. The second benchmark allocates a single structure to write to and read from. The structure used is complex and provides a large variety of read and write operations.

IV. Results and Discussion

This section discusses experiment results. The results of experiment 1 and 2 are presented first, followed by the results of experiment 3.

A. Structure Creation/Allocation without write/read operations

The first benchmark creates and allocates structures whiteout performing write or read operations. The measured execution times and allocation rates of this benchmark across experiment 1 and 2 are presented in Fig. 1 and Fig. 2. They show that LWJGL and FFMA exhibit the best performance across both experiments. The difference between LWJGL and FFMA are relatively small. FFMA is about 1.4 to 1.7 times slower than LWJGL, but both libraries require almost the same amount of Java-Heap memory.

Nevertheless, both outperform the other libraries by more than an order of magnitude in terms of execution time and allocation rate. For example, FFMA is about 13 times faster than LUtils in the first experiment increasing to a factor of 49 in the second experiment. LUtils outperforms JNA in terms of execution time by a factor slightly higher than two in both experiment. In Contrast, LUtils and JNA exhibit almost identical amount of allocated bytes in experiment 1, but in experiment 2 JNA requires about 1.8 times more Java-Heap memory than LUtils.

Overall, when creating and allocating small and medium-sized structures without performing write or read operations LWJGL and FFMA provide the best performance while LUtils and JNA perform about one order of magnitude worse in both execution time and Java-Heap memory allocation.

Most of these results are not unexpected, but the fact that LWJGL's average execution time is slightly lower than the average execution time of FFMA is interesting. This is likely due to the different allocation strategies employed by the libraries. LWJGL is allocating with the native malloc function from jemalloc, while FFMA uses its `Arena` interface to allocate native memory, which provides more safety and correctness guarantees as well as zero-initialised memory.

JNA and LUtils exhibit execution times that are more than six times higher compared to the other evaluated libraries. This behaviour can primarily be attributed to their reliance on reflection and the comparatively complex class hierarchies required to generate structure representations dynamically at runtime. In addition, both libraries perform validation steps each time a structure instance is created, introducing further overhead.

Furthermore, LUtils encapsulates primitive native types within dedicated wrapper classes. This design increases both execution time and allocation rate, as each primitive value is represented by an object rather than a Java primitive type. In contrast, JNA maps native primitive types directly to their corresponding Java primitive types. This is likely the reason why JNA requires slightly less Java-Heap memory in the first experiment, which works with small structures containing mostly primitive types. However, in experiment 2 JNA exhibits an almost two times higher allocation rate than LUtils. This could be explained by more complex runtime validations and initialisations, especially when working with more complex structures.

B. Structure Creation/Allocation with write/read operations

The second benchmark evaluates the creation and allocation of structures including write and read operations to all fields. The execution time and allocation rate measurement results of this benchmark are presented in Fig. 3 and Fig. 4 for both experiment 1 and 2. In addition to the main measurement, the figures include a reference benchmark, which allocates the same structures without performing any write or read operations. Once again, LWJGL achieves the best performance, followed by FFMA, which requires more than four times the execution time and slightly more than twice the Java-Heap memory compared to LWJGL in both experiments. Nevertheless, both libraries outperform LUtils and JNA in both execution time and allocation rate. However, the gap between FFMA and LUtils shrinks compared to the first benchmark. Specifically, the execution time factor between FFMA and LUtils decreases from 13 and 48 in the first benchmark to a factor of 1.5 and 3.6 in benchmark 2. Similar to the first benchmark, JNA displays the worst performance being more

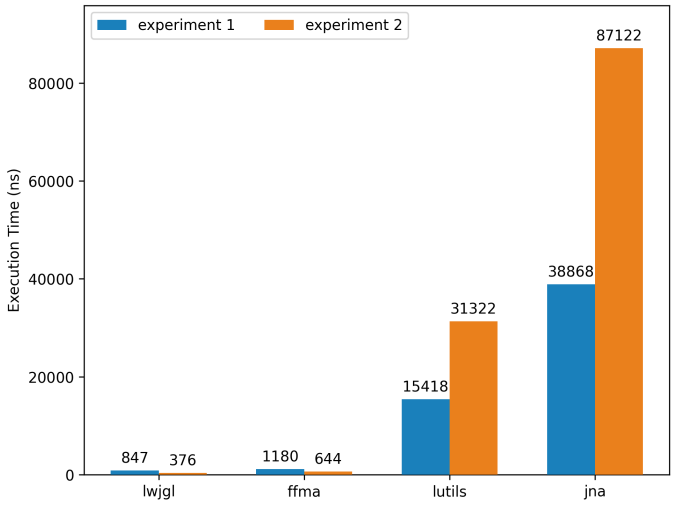

Fig. 1. Execution time results of the first benchmark for experiment 1 and 2. The benchmarks allocate multiple structures without performing write or read operations.

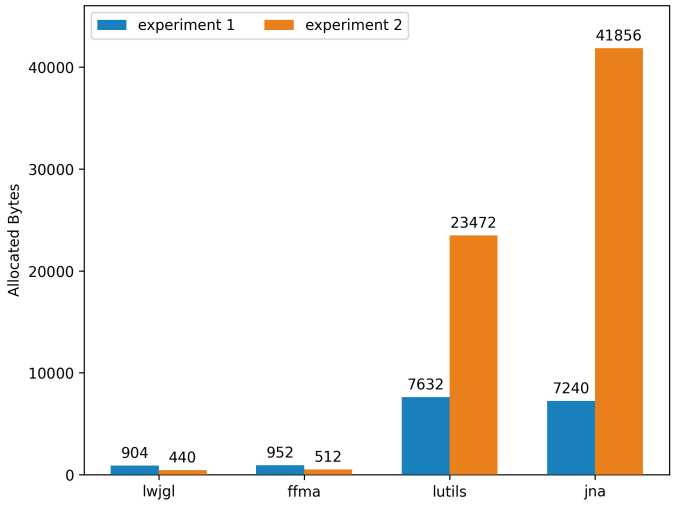

Fig. 2. Allocation rate results of the first benchmark for experiment 1 and 2. The benchmarks allocate multiple structures without performing write or read operations.

than three times slower than LUtils while also exhibiting the highest amount of allocated bytes.

Comparison with the reference benchmark shows that LWJGL and LUtils introduce the smallest absolute write/read overhead in terms of execution time and allocation rate. Their observed execution time overheads range from 101 ns to 429 ns. In contrast, FFMA and especially JNA display higher absolute write/read overheads with execution time overheads ranging from 855 ns to 12,438 ns.

Overall, the results indicate that LWJGL again provides the most efficient implementation for write and read access, both in terms of execution time and memory allocation overhead. FFMA outperforms LUtils and JNA, but displays the largest relative write/read overhead of approximately 80% in both execution time and memory allocation.

Comparing the results to the reference benchmark reveals some interesting observations. Firstly, the large relative write and read overhead of LWJGL is unexpected, because the generated LWJGL structure classes translate most memory operations directly to `Unsafe.put*()` and `Unsafe.get*()` methods, which is a JVM intrinsic and should be replaced with handwritten assembly or bytecode at runtime thereby increasing performance [12]. Looking at the benchmark code, this can be reasonably explained: A large part of the introduced overhead stems from the benchmark code itself, specifically from Integer, Long and Float wrapper classes that are created during the benchmark. Of course, this overhead is introduced to all libraries equally, but the effect is most visible on LWJGL, due to its low reference execution time. Another interesting observation is the large relative overhead of FFMA introduced by writing and reading from the structures. This is likely due to the use of `VarHandle` instead of written or generated functions, which requires additional conversions and checks executed on every write and read operation[5]. LUtils has the smallest relative overhead, because similar to LWJGL it translates write and read operations directly to `ByteBuffer.put()` and `ByteBuffer.get()` methods. LWJGL remains faster, because unlike `Unsafe`, `ByteBuffer` executes additional checks on every write and read operation before calling `Unsafe.put*()` and `Unsafe.get*()`. JNA exhibits the largest absolute overhead, which is likely due to the way the JNA Structures are implemented. The structure fields are represented by normal Java types. Thus, when writing to a structure all values are written to the Java-Heap and only after subsequently calling the `Structure.write()` method, they are read using Java reflection and then converted and written to native memory. Furthermore, reading from the structure using `Structure.read()` requires a similar pipeline.

C. Results for Very Large Structures

The third experiment performs the same benchmarks on very large and complex structures. The execution time and allocation rate results for creating five large structures without performing write or read operations are visible in Fig. 5 and Fig. 6.

Once again, LWJGL displays the best performance across all evaluated libraries. In the first benchmark LWJGL is 30 times faster than the second-best approach, which is LUtils. Meaning that, for the fist time LUtils exhibits a better performance than FFMA by a factor of approximately four. JNA once again displays the worst performance with two orders of magnitude slower execution time than FFMA and LUtils.

The allocation rate results show a slightly different trend. LWJGL and FFMA allocate less than 600 bytes of Java-Heap memory. In Contrast, LUtils requires about two orders of magnitude more Java-Heap memory and JNA again exhibits the highest allocation rate, allocating about 1890 times more bytes than LUtils.

[5]https://github.com/openjdk/jdk/blob/bea48b54e2f423693e1e472129a86b030baf9eee/src/java.base/share/classes/java/lang/invoke/VarHandle.java

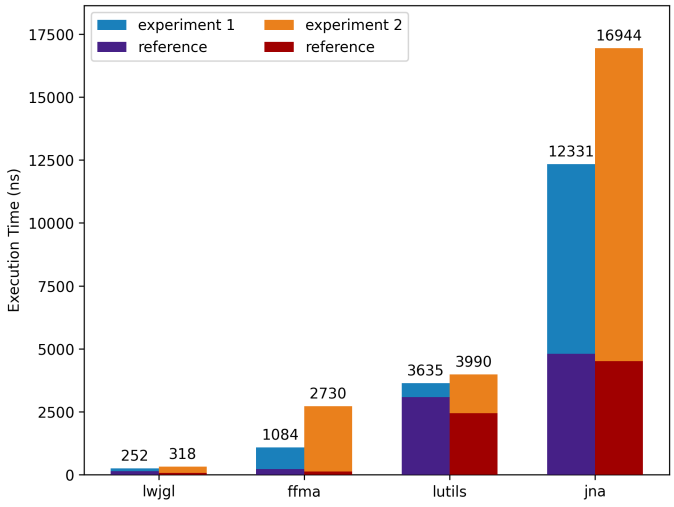

Fig. 3. Execution time results of the second benchmark for experiment 1 and 2. The benchmarks allocate structures and perform write/read operations on all structure fields. Additionally, reference benchmarks are included which allocate the same structures without performing write or read operations, to isolate write/read overhead.

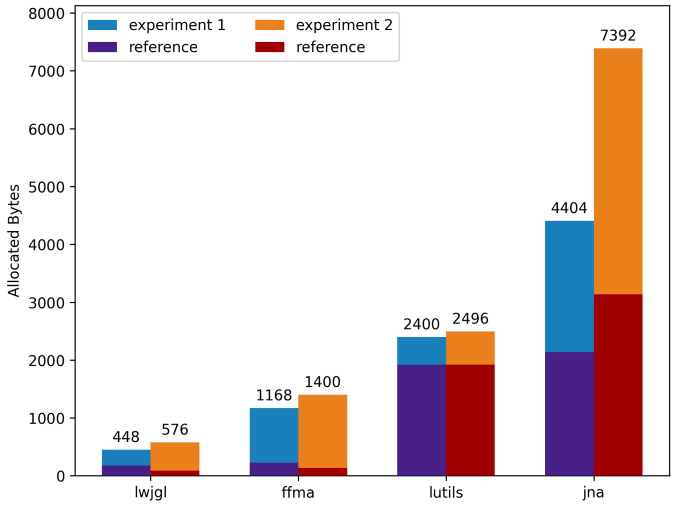

Fig. 4. Allocation rate results of the second benchmark for experiment 1 and 2. The benchmarks allocate structures and perform write/read operations on all structure fields. Additionally, reference benchmarks are included which allocate the same structures without performing write or read operations, to isolate write/read overhead.

The very low execution time of LWJGL is again expected for multiple reasons. Firstly, it does not require any reflective access when the structure is created, and secondly it does not fill the allocated memory with zeros. However, noteworthy is the fact that LWJGL requires as little allocated bytes as FFMA. This is interesting because LWJGL creates classes to easily perform write and read operations on the structures while FFMA does not. These classes should require a large number of allocated bytes due to the high complexity of the nested structures. This disparity can be explained by the fact that the large structures mostly consist of arrays of other structures and arrays of primitive types. LWJGL does not initially create the objects required for each array element. Instead, the objects are only created when required. This means that a large increase in allocated bytes is expected when write and read operations are performed in the write/read benchmark.

The fact that JNA requires 1890 times more Java-Heap memory allocation than LUtils might be explained by two reasons. The first reason is that JNA is the only library, that stores the content of each structure on the Java-Heap before writing it to native memory. Consequently, JNA uses Java primitive type arrays to represent native primitive type arrays. These require more bytes on the Java-Heap based on the length of the array. In Contrast, LUtils provides wrapper classes for these arrays, which require little Java-Heap memory. For example, an array of 100,000 integers requires 400,000 bytes while LUtils requires less than 416 bytes (measured with JMH; contains overhead required for allocating memory). In this benchmark arrays of primitive types are used multiple times in the structures, which amplifies this effect. Furthermore, the structures also contain many arrays of structures, which is the second reason: LUtils creates a Java-Array for each structure array, but like LWJGL it only initialises the elements of the array with structure specific classes when they are required for writing or reading - which is not the case in this benchmark. On the other hand, JNA initialises every array element when the parent structure is created, resulting in an increase of allocation rate.

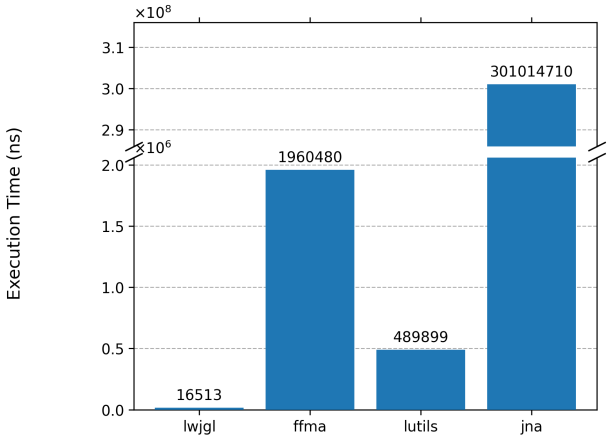

Fig. 5. Execution time results of the first benchmark for experiment 3. This benchmark allocates five large structures without performing write or read operations. Due to the vast differences between JNA and the other libraries, this graph contains a broken axis. This means that the gab between FFMA and JNA is larger than what is visually suggested.

The second benchmark of experiment 3 creates a single large structure and performs write and read operations on all structure fields. The results are summarised in Fig. 7 and Fig. 8. They show that LWJGL, FFMA and LUtils perform similarly for both execution time and allocation rate. LWJGL once again displaying the best performance, followed by LUtils and FFMA, which is about two times slower than LWJGL. Only JNA exhibits a substantially higher execution

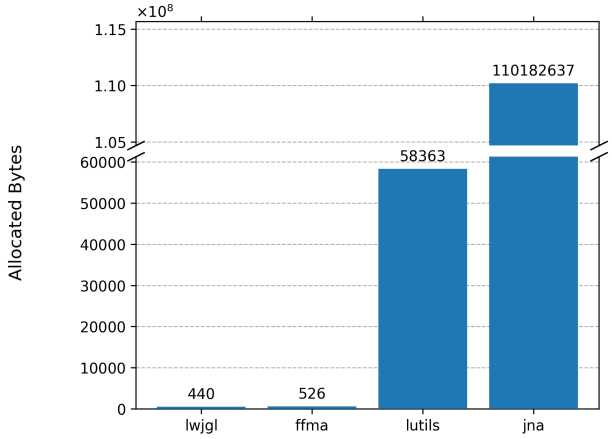

Fig. 6. Allocation rate results of the first benchmark for experiment 3. This benchmark allocates five large structures without performing write or read operations. Due to the vast differences between JNA and the other libraries, this graph contains a broken axis. This means that the gab between FFMA and JNA is larger than what is visually suggested.

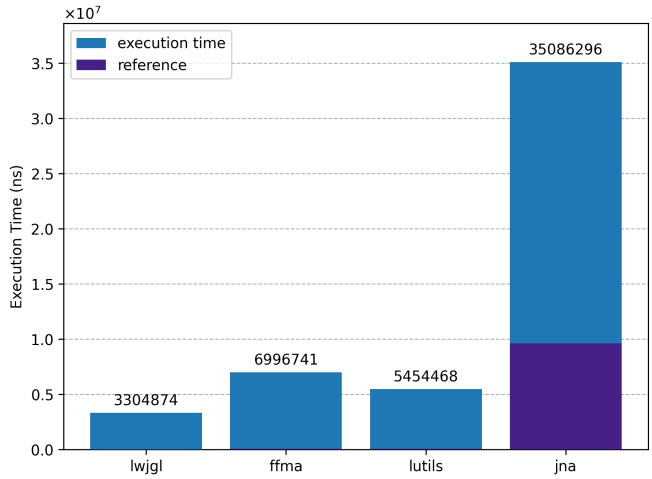

Fig. 7. Execution time results of the second benchmark for experiment 3. This benchmark allocates a single large structure and performs write/read operations on all structure fields. Additionally, a reference benchmark is included, which allocates the same structure without performing write/read operations, to isolate write/read overhead.

time and allocation rate. Its execution time is about five times higher than the other evaluated libraries.

When comparing to the reference values LWJGL, FFMA and LUtils exhibit a relative write/read overhead of more than 98%. Only JNA displays a smaller relative overhead of about 73% compared to its already high reference value. However, JNA also displays the largest absolute write/read overhead.

In this benchmark the large write and read overhead of LWJGL is not as unexpected as in the write/read benchmarks of the previous experiments. The reasoning for that is that LWJGL creates the custom structure classes for each element of all arrays whose component type is not a primitive type. These classes are required to write and read from the elements of the arrays. In fact, this reason also applies to FFMA and LUtils as well. FFMA creates a `MemorySegment` for each element and LUtils creates its custom structure classes extending `ComplexStructure`. Only JNA creates all of its custom structure classes in advance while creating the parent structure, as mentioned during the interpretation of the previous benchmark.

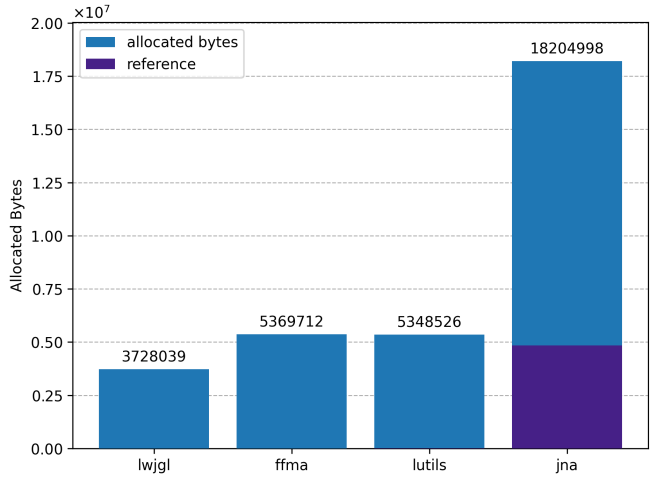

Fig. 8. Allocation rate results of the second benchmark for experiment 3. This benchmark allocates a single large structure and performs write/read operations on all structure fields. Additionally, a reference benchmark is included, which allocates the same structure without performing write/read operations, to isolate write/read overhead.

D. Measurement Variability

The execution time variability of all libraries is relatively low compared to the average values with standard deviations smaller than 10% of their average values across almost all experiments. Only in the first benchmark of experiment 1 JNA displays standard deviations of approximately 125% proportional to its average value, due to 55 outliers reaching values up to 652,340 ns. As a reminder, this benchmark creates ten small structures without performing write/read operations and the average execution time of JNA is 38,868 ns.

The allocation rate variability follows a similar trend. In the first two experiments LWJGL, FFMA and LUtils exhibit negligible standard deviations less than 0.01% relative to their average values. Once again JNA displayed extreme variability in the first benchmark of experiment 1 with a standard deviation of approximately 135% proportional to its average value. Similar to the execution time results, this variability is due to 10 outliers reaching values up to 219,739 bytes. In the other benchmark JNA displays a standard deviation of up to 90 bytes which is still less than 2% compared to the respective average values.

In the third experiment all evaluated libraries showed neglectable standard deviations less than six bytes. Only JNA exhibits higher absolute standard deviations of 146,937 bytes in the first benchmarks and 37 bytes in the second benchmark.

However, proportional to the high average allocation rate, this corresponds to a relative variation of less than 0.2%.

E. Conclusion

Overall, the benchmark results show that LWJGL provides the best performance for allocating structures and write/read operations across different structure sizes. Following LWJGL, FFMA displayed the second-best performance, especially when allocating structures without performing write/read operations. However, FFMA performed worse when working with large and complex structures. JNA generally displayed the worst performance especially when working with larger structures.

LUtils generally performed worse than LWJGL and FFMA but better than JNA. Only in the last experiment – evaluating large and complex structures – LUtils performed better than FFMA in terms of execution time. Furthermore, the results show that LWJGL performed up to 83 times better than LUtils in terms of execution time, suggesting that LUtils should not be used for performance-critical applications without further optimisations.

The preferred choice for performance-critical applications should be LWJGL or FFMA. However, LWJGL shows considerably less write/read overhead, meaning that if the application performs many write and read operations on the structures LWJGL should be preferred. Additionally, if very large structures are required, LWJGL will provide considerably better performance than FFMA.

SUMMARY

This paper introduces the newly developed library LUtils to work with structures using different Application Binary Interfaces (ABI). To determine whether LUtils represents a viable alternative to existing solutions in terms of execution time and memory behaviour, several Java native access libraries were evaluated and compared – namely LWJGL, FFMA, JNA and the introduced library LUtils. To achieve this, multiple JMH (Java Microbenchmark Harness) benchmarks were designed to evaluate the performance of structure creation and write/read operations. These benchmarks measure execution time as well as Java-Heap allocation rate and thereby revealing the following performance characteristics: LWJGL demonstrated the best performance. It is followed by FFMA which displays low execution times for small structures, but shows decreasing performance with large and complex structures. LUtils and especially JNA display generally lower performance, likely due to their use of Java reflection. The evaluation revealed that LUtils should not be used in performance critical applications like real-time rendering, because LWJGL provides up to 83 times better performance. Possible candidates for performance critical applications include LWJGL and FFMA. However, LWJGL consistently displays better performance than FFMA, especially when working with very large structures or when performing many write/read operations.

REFERENCES

[1] S. Liang, “The Java native interface: programmer’s guide and specification,” Addison-Wesley Professional, 1999.

[2] A. Leonard, “Java Coding Problems: Become an expert Java programmer by solving over 250 brand-new, modern, real-world problems,” Packt Publishing Ltd, 2024.

[3] F. Li, et al. “Improving memory performance of embedded Java applications by dynamic layout modifications,” in 18th International Parallel and Distributed Processing Symposium, IEEE Proceedings, 2004.

[4] G. Tan, et al. “Safe Java Native Interface,” in ISSSE, 2006.

[5] M. Grichi, et al. “State of practices of java native interface,” in Proceedings of the 29th Annual International Conference on Computer Science and Software Engineering, 2019.

[6] G. Tan and J. Croft, “An Empirical Security Study of the Native Code in the JDK,” in Usenix Security Symposium. 2008.

[7] N. Tomilov and V P. Turov, “Evaluating the performance of Java Vector API in vector embedding operations,” in Computing Telecommunication and Control vol. 17 issue 4, pp.7-15, 2024.

[8] V. Sochat and T. Haines, “Binary-level Software Compatibility Tool Agreement,” arXiv preprint, arXiv:2212.03364, 2022.

[9] N. Seppälä, “Improving Java Performance With Native Libraries,” bachelor thesis at Metropolia University of Applied Sciences, 2024.

[10] R. Marcelino, B. Combemale, and B. Baudry, “Automatic microbenchmark generation to prevent dead code elimination and constant folding,” in Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering, 2016.

[11] H. Karer and P. Soni, “Dead code elimination technique in eclipse compiler for Java,” in International Conference on Control, Instrumentation, Communication and Computational Technologies (ICCICCT), IEEE, 2015.

[12] B. J. Evans, J. Gough, and C. Newland, “Optimizing Java: practical techniques for improving JVM application performance,” O’Reilly Media Inc., 2018.

[13] L. Traini, V. Cortellessa, D. D. Pompeo, M. Tucci, “Towards effective assessment of steady state performance in Java software: Are we there yet?,” Empirical Software Engineering 28.1, 2023.